

CSC 562 PA2 Maze

This project implements a high-performance maze-solving application capable of processing very large mazes, from 10×10 up to $20K \times 20K$.

My goal was to create a multi-threaded solver that performs better than the provided single-threaded versions by dividing the workload across threads and letting the search progress simultaneously from both ends of the maze.

The multi-threaded design uses two DFS-based solvers:

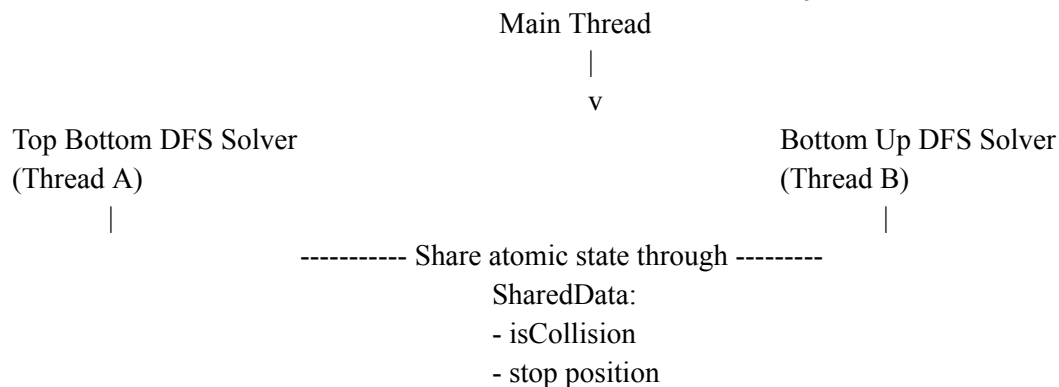
Top solver: starts at the maze's entrance (top, start position).

Bottom solver: starts at the maze's exit (bottom, end position).

Both threads explore the maze concurrently using DFS and communicate through a shared atomic state that lets them detect when they collide (meet somewhere in the middle), and signal that a solution has been found, so the other thread can stop.

The main idea is that each thread only needs to explore roughly half of the maze before the searches either meet or one thread finishes, which can significantly reduce the total search time compared to a single DFS from one end.

The main thread owns the maze and creates the two DFS worker objects and the shared state:



This is done by:

```
TopDown_STMazeSolverDFS topSolver(pMaze, &sharedData);
BottomUp_STMazeSolverDFS bottomSolver(pMaze, &sharedData);
```

```
std::vector<Direction>* pTopPath = nullptr;
std::vector<Direction>* pBottomPath = nullptr;
```

```
std::thread tTop([&]() { pTopPath = topSolver.Solve(); });
std::thread tBottom([&]() { pBottomPath = bottomSolver.Solve(); });
```

```
tTop.join();  
tBottom.join();
```

Both solver threads receive a pointer to the same `SharedData` instance, which they use for cross-thread signaling.

`SharedData` (shared between both threads):

```
std::atomic<Position> stop; – meeting / stopping position  
std::atomic_bool isCollision; – boolean “stop” flag (false → keep searching, true → some event  
happened)  
manual padding bytes to satisfy alignment.
```

All accesses to this state use `memory_order_relaxed`, because I only need “happens-before” style visibility of the flag and the stop position, not total ordering of all operations.

The top-down solver begins its search from the maze’s start position and explores using a depth-first strategy with a fixed move ordering. As it progresses, every cell it visits is marked using a top-specific visitation bit.

On every step, the top thread checks two things:

Global stop flag – if `sharedData.isCollision.load(std::memory_order_relaxed)` is true, it immediately stops exploring and unwinds its DFS. Local event at the current cell – it checks whether the current cell has already been visited by the bottom solver. If so, the two searches have met in the middle.

When the top solver detects either: a collision with the bottom thread’s path, or that it has reached the maze’s end position and found a complete path by itself, it sets the shared flag and stop position:

```
sharedData.isCollision.store(true, std::memory_order_relaxed);  
sharedData.stop.store(currentPos, std::memory_order_relaxed);
```

This tells the other thread to stop as soon as it next checks `isCollision`, and it records a stopping point that can be used to stitch paths together.

Bottom-up solver section (same idea)

The bottom-up solver starts from the maze’s exit and runs a similar DFS from the bottom toward the entrance. It uses a complementary visitation bit (via an `isBottomCollision()` style function) to mark cells it has visited.

During its traversal, the bottom solver also repeatedly:

checks the `isCollision` flag and stops if another event has already been signaled, and tests the current cell for either having been visited by the top solver or being equal to the maze’s start position. If the bottom solver reaches the start while the top has not signaled anything yet, or steps onto a cell previously marked by the top solver, it sets `sharedData.isCollision` to true and stores the current position in `sharedData.stop`.

From this point on, both threads will eventually stop, and each solver returns a partial path vector to the main coordinating code.

After both worker threads have joined, the main solver (MTMazeStudentSolver::Solve) inspects the results. It normalizes pTopPath and pBottomPath so that neither is nullptr (if a solver returns nullptr, it is replaced with an empty std::vector<Direction>).

It loads the boolean flag:

```
const bool collided =  
    sharedData.isCollision.load(std::memory_order_relaxed);
```

It allocates a new result vector:

```
auto* pResult = new std::vector<Direction>();
```

If collided is true, we assume both threads produced a partial path:

pTopPath holds directions from start → meeting point,

pBottomPath holds directions from meeting point → end (or an equivalent suffix path).

The final answer is built by concatenating both:

```
pResult->reserve(pTopPath->size() + pBottomPath->size());
```

```
pResult->insert(pResult->end(), pTopPath->begin(), pTopPath->end());
```

```
pResult->insert(pResult->end(), pBottomPath->begin(), pBottomPath->end());
```

If collided is false, then no explicit cross-thread meeting was recorded. In that case, the solver falls back to whichever side successfully found a full path. In all cases, the temporary pTopPath and pBottomPath vectors are deleted, and the heap-allocated pResult is returned to the caller as the final solution.

Memory visibility is guaranteed because all shared variables are atomic. Writes to isCollision become visible immediately; The stop position is also visible without racing conditions, and there are no fences, locks, or memory barriers involved.

At first, I thought about using more than 2 threads to do more work concurrently, but that was met with its own challenges of deciding on the division of workload. More precisely, I couldn't think of a good system to have more than the 2 threads (1 thread going from the entrance and the other from the exit). I ran into the problem of having to decide how to glue the solution together if I had multiple collisions from more than 2 threads.

I learnt that atomics are ideal for simple cross-thread signaling, which lines up with what we learnt in class. I also learnt that it is crucial to have a way for the threads to communicate. In this project, SharedData made communication clean and safe. I also think that understanding memory ordering and atomics is essential for C++ concurrency.